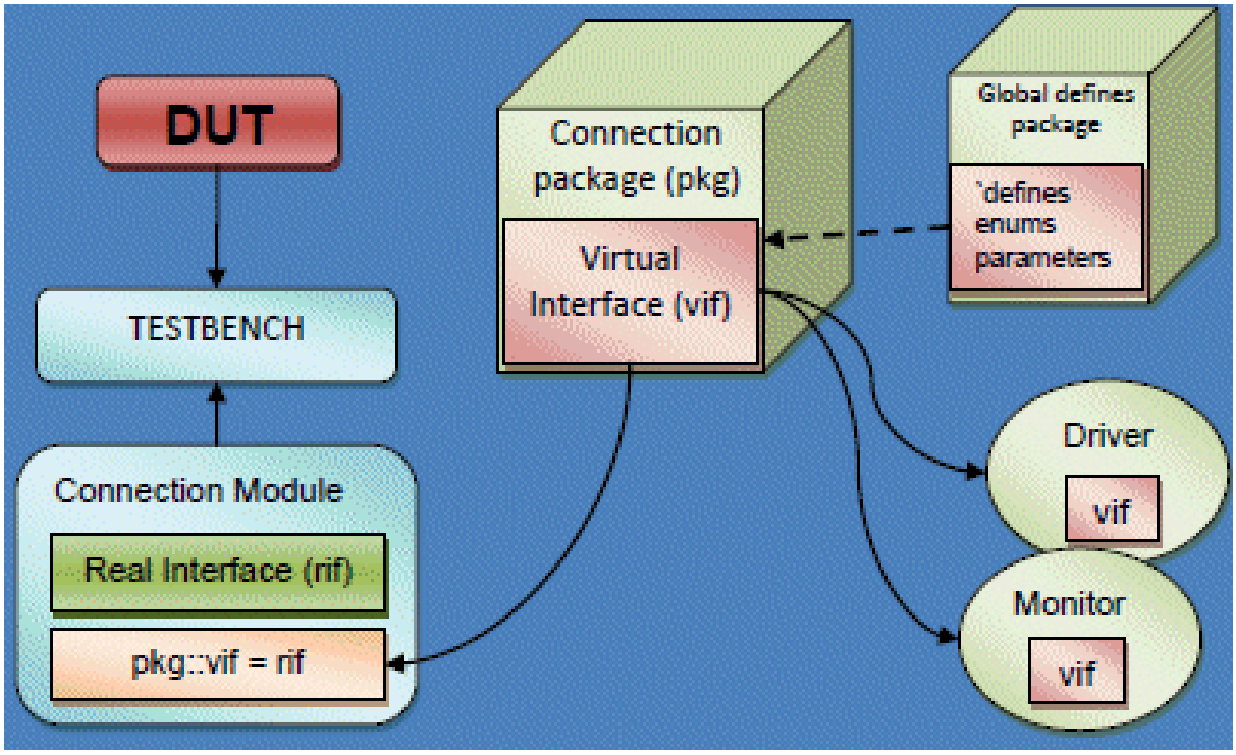
LAB 11: Virtual Interface

Date:4 July of 2025



Proffesor: Dr. Miguel Angel Rivera Acosta

Students: **Luis Alberto Castañeda Montaño**

José Antonio García Madrigal

Cesar Eduardo Inda Ceniceros

Alonso Emmanuel López Macías

Team: 9

Introduction

In this laboratory, the use of **virtual interfaces** in SystemVerilog was explored as a key mechanism for connecting verification classes with synthesizable modules through an object-oriented environment. The Design Under Test (DUT) implements a simple 32-bit adder module, which performs a synchronous addition operation and validates its result through a control signal **(ivalid)**. The **adder_interface** encapsulates the signals that connect the DUT with the testbench, grouping inputs and outputs into a single reusable entity. Based on this interface, a class named **adder_stimulus_class** was developed to generate randomized test stimuli, constrained by specific limits, and transmit them to the DUT via a **virtual interface**.

The use of virtual interfaces in SystemVerilog is fundamental in modern verification environments, as it enables abstraction and decoupling between the stimulus logic (verification classes) and the physical design signals. This not only enhances the reusability and maintainability of the testbench, but also facilitates the development of modular and scalable verification environments. Additionally, it allows multiple instances of the same verification class to interact with different interfaces without rewriting code, supporting the simulation of complex and hierarchical designs.

In summary, this practice introduces the concept of a virtual interface as a bridge between the structured paradigm of hardware design (RTL) and the object-oriented paradigm of functional verification, laying the groundwork for professional methodologies such as UVM (Universal Verification Methodology).

The use of virtual interfaces is not just an advanced technique, but a practical necessity for scaling testbenches toward high-complexity digital systems. This lab marks an important step toward transitioning from structured test environments to professional verification methodologies like **UVM, where reusability, abstraction, and object-oriented design** are essential.

In this practice, the following actions were carried out to demonstrate the functionality of **virtual interfaces**:

1. The test logic was encapsulated within a class (**adder_stimulus_class**) without direct access to physical signals.
2. The **std::randomize ()** function was used with constraints to generate valid operand values.
3. A real interface instance **(adder_interface)** was assigned to a virtual interface pointer (**vif**) within the class.
4. Stimulus tasks were invoked through the virtual interface as if they were part of the class, demonstrating the power of abstraction.

Requirements

- Compile & run the program in ***"Vivado".***
- Test operations for **"task's, stimulus, constraints & functions".**
- Validation for **"task's, stimulus, constraints & functions".**

Development

In the code we modified the section for some task methods and add a line **$display** for see the values in the console.

```
task task_initialize();
    vif.initialize();
endtask

task task_drive_rand_data();
    this.randomize();
    $display("[RAND DATA] a = %0d, b = %0d, max = %0d", a, b, max); //!Line added
    vif.drive_data(this.a, this.b);
endtask

task task_drive_100_rand_data();
    repeat(100) begin
        this.randomize();
        $display("[100 DATA] a = %0d, b = %0d, max = %0d", a, b, max); //!Line added
        vif.drive_data(this.a, this.b);
    end
endtask

function func_set_max_value(int max);
    this.max = max;
endfunction
```

Figure: 1 Modification1

Other modification that we did in the “testbench” → Initial Begin we added some stimulus and stimulus condition for validation in the constraints code and see that the result would be correctly.

```
    initial begin

        add_stimulus = new(); // creates an object of adder_stimulus_class type
        add_stimulus.vif = intf; // assign the intf (adder_interface handle) to the add_stimulus object vif member
        @(posedge rstn);
        add_stimulus.task_initialize();
        repeat(10) begin
            repeat($urandom_range(1,5)) @(posedge clk);
            add_stimulus.task_drive_rand_data();
        end

        repeat($urandom_range(10,50)) @(posedge clk);
        add_stimulus.task_drive_100_rand_data();
        repeat($urandom_range(10,50)) @(posedge clk);
        add_stimulus.func_set_max_value(100);
        add_stimulus.task_drive_100_rand_data();
        repeat($urandom_range(10,50)) @(posedge clk);
        repeat(10) @(posedge clk);
        $finish;

        //todo: test this section code
        // Establecer un nuevo límite para a y b
        add_stimulus.func_set_max_value(100);
        $display("=== Generating 100 random transactions with max = %0d ===", 100);

        repeat(100) begin
        add_stimulus.randomize();

        // Verificación visual
        $display("[CHECK] a = %0d, b = %0d, max = %0d", add_stimulus.a, add_stimulus.b, add_stimulus.max);

        // Validación automática de constraint
        if (add_stimulus.a > add_stimulus.max || add_stimulus.b > add_stimulus.max) begin
            $error("ERROR: Constraint violated! a = %0d, b = %0d, max = %0d",
                   add_stimulus.a, add_stimulus.b, add_stimulus.max);
        end

        // Enviar datos al DUT
        add_stimulus.vif.drive_data(add_stimulus.a, add_stimulus.b);
        end
        //todo: test this section code
    end

endmodule
```

Figure: 2 Modification2

Vivado

For the purpose of this practice, we chose the **ZCU104** in order to generate the simulation and project components through the selection of boards.

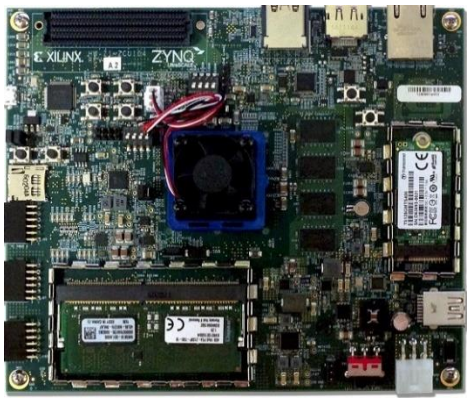


Figure: 4 ZCU104_Board

We create 2 folders one of them:

documentation → for add the information relevant in the laboratory.

rtl → for main design.

dv → for testbench's & simulations

TestBench

In this case, the behavior in the **console** was as follows.

```
Time resolution is 1 ps
[RAND DATA] a = 3409799080, b = 3242061410, max = 4294967295
[RAND DATA] a = 1037303635, b = 4056594817, max = 4294967295
[RAND DATA] a = 200792494, b = 2299619067, max = 4294967295
[RAND DATA] a = 3837096602, b = 4208730236, max = 4294967295
[RAND DATA] a = 2262039082, b = 3679820963, max = 4294967295
[RAND DATA] a = 3951652036, b = 322155791, max = 4294967295
[RAND DATA] a = 3336505704, b = 2296761549, max = 4294967295
[RAND DATA] a = 2276019079, b = 3658661495, max = 4294967295
[RAND DATA] a = 21555902, b = 3807939396, max = 4294967295
[RAND DATA] a = 29131807, b = 3936474828, max = 4294967295
[100 DATA] a = 3958030730, b = 3988359748, max = 4294967295
[100 DATA] a = 546675154, b = 3987162537, max = 4294967295
[100 DATA] a = 2911135702, b = 3680554989, max = 4294967295
[100 DATA] a = 2641084703, b = 4227230143, max = 4294967295
[100 DATA] a = 695914902, b = 4160024884, max = 4294967295
[100 DATA] a = 3194544851, b = 3336999605, max = 4294967295
```

Figure: 3 Console Values

In the run simulation behavioral (waveform):

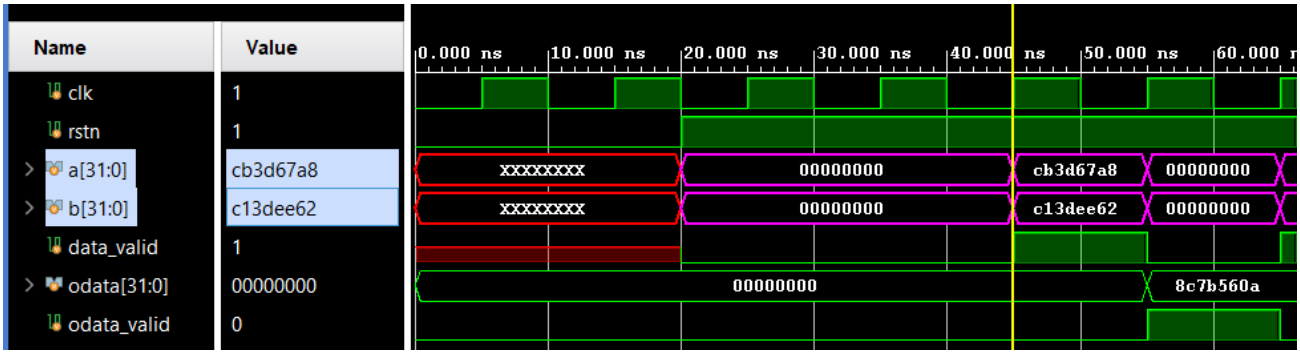


Figure: 4 Waveform Behavioral

GitHub Repository

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA_5/tree/Lab11_VirtualInterface

Conclusion / Issues

José Antonio Garcia Madrigal

In this practice, we gained an understanding of how virtual interfaces work in SystemVerilog, which allow object-oriented classes to interact indirectly with the physical signals of the design (DUT). This abstraction is essential for building more flexible, reusable, and scalable testbenches. We learned to maintain a clear separation between the design logic (RTL) and the stimulus/verification logic. By encapsulating the signals within an interface **(adder_interface)** and referencing it from a class **(adder_stimulus_class)** using a **virtual interface (vif),** we improved the maintainability and reusability of the testbench.

Alonso Emmanuel Lopez Macias

Constraints were implemented within the verification class to limit the random values generated by **std::randomize(),** demonstrating how to generate valid stimuli automatically. This is especially useful in constrained-random functional verification. The practice allowed us to apply stimuli to the adder module in a structured manner, verify its behavior under different input conditions, and observe that the outputs were consistent with expectations. An automatic validation mechanism was also integrated to detect any violations of the defined constraint.

Luis Alberto Castañeda Montaño

The use of virtual interfaces is a core technique in professional methodologies such as **UVM (Universal Verification Methodology)**. This practice provides a solid foundation for adopting modern verification techniques, where object-oriented programming, code reuse, and decoupling are essential principles.

Cesar Eduado Inda Ceniceros

I understood that in real verification environments, especially for ASICs and FPGAs, engineers work with highly complex designs that require modular, scalable, and automated verification. Virtual interfaces allow drivers, monitors, and other testbench components to access **DUT** signals without breaking the principle of encapsulation. Learning this at an early stage also helps accelerate the transition toward professional frameworks like **UVM**, which are now industry standards in the semiconductor field.